
NEXTXR DIGITAL TWIN

Digital Twin Features

108 features across the platform — what is built, what needs porting, what is not started

Generated 7 September 2026, from the running system

Repository Tejesh3305/Goalcert_Digital-Twin

Source docs/DIGITAL-TWIN-FEATURES.md

Status Reflects the deployed implementation, not a design proposal

The target. Workforce Intelligence becomes the main application: an organisation signs in, every person lands in the workspace for their role, and the digital twin, the simulation engine and the agentic AI are capability surfaces reached *through* that workspace at the access level their role allows.

What this document is. Every feature that platform needs, with an honest status against two codebases:

	Meaning
☒ Built	Working in the digital twin (<code>nextxr-ontology-v3</code>) today
☐ Not built	Not in the digital twin — whether it exists elsewhere or nowhere

The status is about the digital twin and nothing else. A feature that works in `workforce-intelligence` but not here is Not built, because it is not in this product yet.

Where such a feature already exists is kept in its Notes column, since it changes what the work is — porting rather than inventing — but it does not change the status.

Surveyed 2026-09-07 against `nextxr-ontology-v3` (27 tables, 188 API routes) and `workforce-intelligence` (15 work tables, 7 roles collapsed to 5 here, 6 role applications, a 9-domain scenario engine).

1. Identity, hierarchy and access

The platform's spine: one account model, a role per person, and every surface rendered from that role's capability list.

#	Feature	Status	Notes
1.1	Organisations, users, sign-in, sessions	✓ Built	JWT + rotating refresh, session revocation, logout, MFA field
1.2	API keys, scoped per tenant	✓ Built	Hashed, role-carrying, revocable
1.3	Audit log of account actions	✓ Built	<code>audit_log</code> , 190 rows live
1.4	Goalcert Hub SSO	✓ Built	Ticket exchange with replay guard (<code>hub_sso_jti</code>)
1.5	Data role: owner / admin / write / read	✓ Built	<code>memberships.role</code> , enforced in <code>tenancy.py</code>
1.6	Five-role hierarchy	○ Not built	Twin has two personas (supervisor, frontline). The platform needs five — see below
1.7	Capability matrix per role	○ Not built	Twin's <code>work/authority.py</code> covers dispatch only. The five roles need ~20 capabilities across content, clearance, evidence, KPI and org
1.8	Team scoping ("whose people?")	✓ Built	<code>assignable_user_ids</code> — supervisor is authoritative over their own team only
1.9	Frontend renders from the capability list	✓ Built	<code>GET /work/me</code> returns capabilities; buttons drawn from it
1.10	Approval routing by role	○ Not built	<code>APPROVER_FOR</code> routes each approval kind to the right inbox — retargets to Manager / CEO
1.11	Multi-tenant isolation	✓ Built	<code>org_tenants</code> + per-label <code>(tenantId, id)</code> graph constraints

The five roles

One line of command plus one platform role, which is all the hierarchy this product needs:

```
Admin • platform: provisioning, connectors, health – not in the line
```

```
CEO → Manager → Supervisor → Frontline operator
org   site /      one team    the job in front
wide  function
      of them
```

Role	Owns	Core capabilities
Frontline operator	The job in front of them	act on own assignments, act on work orders, request assist
Supervisor	One team	create assignments (own team), create/close work orders, respond to assist, approve clearance exceptions, read team
Manager	A site or function, several teams	everything a supervisor has but across their teams, plus author/review/publish content, revoke clearances, read and export evidence, read agent logs
CEO	The organisation	set KPI targets, read evidence and readiness org-wide, manage case studies, create assignments
Admin	The platform, not the shift	org management, connector management, platform health

This is a deliberate simplification of what WI ships. WI has seven roles: it splits `lnd` (authoring) and `compliance` (clearance and evidence) into their own roles, and separates `coo` from `super_admin`. Collapsing to five folds **L&D and compliance into Manager**, maps `coo` → **CEO**, and merges `super_admin` into **Admin**.

The capabilities do not disappear — they are redistributed. Porting is therefore a re-keying of WI's **CAPABILITIES** dict rather than a redesign, and the two capabilities that were split across `lnd` and `compliance` (`content.publish` , `clearance.revoke`) simply land on the same role.

Worth knowing before you commit to five. The one thing the merge gives up is *separation of duties*: with Manager holding both `content.publish` and `clearance.revoke` , the person who authors a procedure can also clear people on it and revoke the clearance. In a regulated audit that is usually a finding. It is entirely reasonable to accept for now — the fix later is one extra role, not a redesign, because the capability matrix is data. Flagging it so the choice is made rather than inherited.

The gap that matters here is 1.6/1.7, and everything else hangs off it. The twin's persona axis was built for a two-role dispatch loop. Five roles and a wider capability matrix should be done first; most features below gate on it. The twin's existing two-axis design — `role` for data access, `persona` for job — extends to five personas without structural change.

Also worth keeping: a `role='read'` supervisor currently cannot dispatch, because the auth middleware refuses every POST from a read-only principal before the persona check runs. With five roles that inconsistency gets more expensive, not less.

2. Per-person dashboards

Each role lands in its own workspace. Five roles, five applications — WI already ships six shells, so this is a merge rather than new construction.

#	Application	Role	Status	What it is
2.1	Field	Frontline operator	✔ Built	Today's jobs, the procedure, the fix. The twin's is the better of the two
2.2	Command	Supervisor	✔ Built	Dispatch queue, team load, sign-off. Built in the twin
2.3	Manage	Manager	○ Not built	Merge of WI's Studio (authoring, review, publish) and Evidence (clearances, evidence packs, export, agent logs), plus multi-team dispatch
2.4	Exec	CEO	○ Not built	KPIs, targets, readiness, case studies, org-wide rollup
2.5	Control	Admin	○ Not built	Org provisioning, connectors, platform health
2.6	Role-based routing, lazy-loaded per app	○ Not built	PersonaRouter — a session downloads only its own application	
2.7	"No workspace for this role" handling	✔ Built	Both treat an unassigned role as a provisioning bug, not a crash	
2.8	Operator dashboard with charts and live 3-D	✔ Built	KPI tiles, completion bars, severity mix, score trend, XP, and the job's twin rendered beside it	
2.9	Supervisor team-load view	✔ Built	Per-operator load and throughput, org-wide with assignable flags	
2.10	Manager multi-team view	○ Not built	The twin's team view is single-supervisor scoped; a Manager spans several teams	

Reading of this section: the two operational workspaces are built, and the twin's are ahead of WI's — charts, a live 3-D asset panel, click-through to the plant. The three management workspaces exist only as WI shells. The work is porting the twin's dashboard standard into them, not building dashboards from nothing.

On merging Studio and Evidence into one Manage app: they were separate in WI because they served separate roles. Under five roles one person does both, and two applications for one person is two doors to the same room. Manage should be one workspace with sections, not a tab bar joining two former apps.

3. Work: dispatch and execution

#	Feature	Status	Notes
3.1	Teams and team membership	✓ Built	
3.2	Assign work to a person, team-scoped	✓ Built	Enforced at service and route level; tested
3.3	Raise work by hand (not from a detection)	✓ Built	POST <code>/work/tasks</code> , exposed in Dispatch
3.4	Task lifecycle: open → assigned → in progress → resolved → closed	✓ Built	Legal transitions enforced in one module
3.5	Blocked state with a reason	✓ Built	
3.6	Append-only transition history	✓ Built	<code>task_events</code> — answers "who assigned this, when"
3.7	Operator inbox, own work only	✓ Built	No user id parameter; server reads the session
3.8	Supervisor sign-off closes the loop	✓ Built	Closing resolves the Finding back in the graph
3.9	Assignment and WorkOrder as separate objects	○ Not built	Twin merges both into <code>tasks</code> . WI models them separately — an assignment is <i>what you owe</i> , a work order is <i>the authorised job</i>
3.10	Assist: request help, respond, close	○ Not built	<code>AssistSession</code> — an operator escalates to a supervisor mid-job
3.11	Notifications	○ Not built	WI has the <code>Notification</code> table and endpoints; no delivery channel exists in either (no email/push/SMS)
3.12	Work rules: which findings become work	✓ Built	Data-driven; default critical-only, dedup by finding node
3.13	Auto-generated work order content	✓ Built	AI-written, on explicit request

4. Competency, clearance and evidence

The largest genuinely missing area in this repo. Nothing here exists in the twin; all of it exists in WI.

#	Feature	Status	Notes
4.1	Clearance: a person is cleared to do a procedure on an asset class	○ Not built	<code>Clearance</code> model
4.2	Evidence links backing a clearance	○ Not built	<code>EvidenceLink</code> — what proves competency
4.3	Clearance revocation	○ Not built	Manager capability under the five-role model
4.4	Clearance exception approval	○ Not built	Routed to Supervisor or Manager
4.5	Evidence pack export	○ Not built	The audit deliverable
4.6	Clearance gates dispatch	○ Not built	Neither codebase refuses an assignment because the assignee is not cleared. This is the point of the feature and it is unbuilt in both
4.7	Expiry and recertification	○ Not built	No scheduled re-validation anywhere
4.8	Agent action log for audit	○ Not built	Twin meters agent spend and records <code>task_events</code> ; WI has <code>agentlog.read</code> as a capability. Neither writes a reviewable per-decision agent log

4.6 is the one to notice. Clearance data without a gate is a record nobody is forced to consult. The moment `assign` refuses an uncleared operator, the whole competency model starts paying for itself — and that check belongs next to `assignable_user_ids` , which already exists in the twin.

5. Learning, content and the improvement loop

#	Feature	Status	Notes
5.1	Procedure catalogue	○ Not built	Twin has 980 lines of hand-authored procedures in code ; WI has a Procedure table. Content in code cannot be authored by an L&D user
5.2	Graded decision drills with server-side scoring	✓ Built	Answer key never leaves the server; tested
5.3	Practice vs assessed runs	✓ Built	
5.4	Hints, wrong-step penalties, debrief	✓ Built	
5.5	XP, levels and an auditable ledger	✓ Built	Twin-only — WI has no XP model
5.6	Training library with per-item standing	✓ Built	Passed / in progress / attempts / last score; resume a half-finished run
5.7	"Repair with AI" reusable trainer	✓ Built	Component reachable from the job, the library and the twin
5.8	Candidate content pipeline: draft → review → publish	○ Not built	CandidateContent + content.publish / content.review
5.9	Authoring UI for procedures	○ Not built	WI's Studio app; becomes a section of Manage
5.10	Loop events: what the fleet learned	○ Not built	LoopEvent — feeding outcomes back into content
5.11	Curriculum / learning paths	○ Not built	No ordered multi-procedure programme in either
5.12	Skills matrix per person	○ Not built	Implied by clearances; no aggregate view

6. Scenario builder and simulation engine

You asked specifically for the scenario builder. It exists — in WI, not here.

#	Feature	Status	Notes
6.1	Scenario builder / authoring studio	○ Not built	<code>scenario_engine/api/studio.py</code> — authors a scenario from a natural-language description , browses domains and fault types, runs it
6.2	Scenario model: objectives, steps, decision gates, target selectors	○ Not built	<code>engine/scenario.py</code>
6.3	Monte Carlo runs	○ Not built	<code>engine/monte_carlo.py</code> — distribution of outcomes, not one path
6.4	World model: actors, resources, environment	○ Not built	<code>engine/world.py</code> , <code>environment.py</code>
6.5	Workflows and conditions	○ Not built	<code>engine/workflows.py</code> , <code>conditions.py</code>
6.6	KPI evaluation per run	○ Not built	<code>engine/kpis.py</code>
6.7	Nine domain plug-ins	○ Not built	aerospace, defence, edm, ev, facility, fleet, hospital, railway, solar
6.8	Run manager, run history, tripwires	○ Not built	<code>api/runs.py</code> , <code>api/tripwire.py</code>
6.9	Scenario reports	○ Not built	<code>reports/generator.py</code>
6.10	Guided mode	✅ Built	Both have a guided path; the twin's is the graded operator drill
6.11	Scenario engine reads live twin state	○ Not built	<code>services/twin_client.py</code> — the seam that makes scenarios run against a real plant
6.12	Physics simulation of a machine	✅ Built	Different thing, and the twin is ahead: <code>dynamics/</code> integrates wear, faults persist, one lease-owner per tenant ticks it

Two different simulators, and they are complementary rather than duplicated. The twin's `dynamics/` engine answers *what the machine does* — a physical integrator where wear accumulates and an injected fault persists. WI's `scenario_engine` answers *what the organisation does about it* — actors, resources, decision gates and a Monte Carlo spread of outcomes. The platform needs both, wired together through `twin_client.py` . Neither replaces the other, and the deliberate decision not to port WI's engine into the twin still stands; what changes is that WI becomes the host, so the engine is already in the right place.

7. The digital twin as a capability surface

This is the area the twin is furthest ahead on, and none of it exists in WI's older embedded copy at the same standard.

#	Feature	Status	Notes
7.1	Ontology-driven graph: 11 node categories, SHACL shapes	✓ Built	
7.2	Entity CRUD with a hash-chained change log	✓ Built	Tamper-evident
7.3	Live telemetry ingest (device tokens, bulk, idempotent)	✓ Built	Composite PK makes replay safe
7.4	Protocol connectors + SunSpec discovery	✓ Built	14 endpoints
7.5	Historian: history, trends, signal discovery	✓ Built	34,347 measurements live
7.6	Behaviour registry raising Findings	✓ Built	
7.7	Live 3-D scenes per domain	✓ Built	Including a turbine model and a hospital floor plan
7.8	Photo → 3-D reconstruction	✓ Built	Via RunPod GPU
7.9	Twin builder from natural language	✓ Built	Agent-driven
7.10	Per-tenant runtime ownership via Redis lease	✓ Built	Prevents duplicate writes at 2+ tasks
7.11	SSE event bus per tenant	✓ Built	
7.12	Prediction / RUL and cascade analysis	✓ Built	
7.13	Twin access scoped by workforce role	○ Not built	Today twin access is <code>role + org_tenants</code> . Nothing expresses "a frontline operator may see only the assets their team owns"
7.14	Asset ↔ twin binding from the work side	○ Not built	WI's <code>POST /work/assets/bind-twins</code> ; the twin has no equivalent
7.15	Graph is populated	○ Not built	3 nodes against 16 registered twins. Schema and constraints are provisioned; content is not. Everything above is demonstrable only once this is seeded

8. Agentic AI

#	Feature	Status	Notes
8.1	16 embedded copilot agents	✔ Built	Narration, diagnosis, analysis, cascade, work order, procurement, incident report, procedure, troubleshoot, chat
8.2	Deterministic stub for every agent	✔ Built	Keyless operation never silently fakes a real answer
8.3	Spend ledger, response memo, in-flight coalescing	✔ Built	Per-model token and cost accounting
8.4	Per-tenant budget cap	○ Not built	Implemented but disabled unless <code>NXR_COPILOT_BUDGET_USD</code> is set
8.5	Long-running agent sessions	✔ Built	26 endpoints: twin builder, bundle author, plugin, accelerator, ops
8.6	Scenario analyst agent	○ Not built	<code>embedded_agents/scenario/analyst.py</code>
8.7	Scenario authoring agent	○ Not built	<code>embedded_agents/scenario/authoring.py</code> — powers the builder
8.8	Per-role agent access levels	○ Not built	Every signed-in user can call every agent. No role gates which agents run, and no per-role budget
8.9	Reviewable agent decision log	○ Not built	Spend is metered; individual decisions are not recorded for audit

9. Analytics, KPIs and readiness

#	Feature	Status	Notes
9.1	Operator's own stats: counts, series, score, streak	✓ Built	<code>GET /work/stats/me</code>
9.2	Supervisor team throughput	✓ Built	<code>GET /work/team-stats</code>
9.3	XP ledger and standing	✓ Built	
9.4	KPI points and targets	○ Not built	<code>KpiPoint</code> , <code>kpi.set_target</code>
9.5	Readiness snapshots	○ Not built	<code>ReadinessSnapshot</code> — is the workforce ready for the work
9.6	Org-wide executive view	○ Not built	Exec app
9.7	Case study management	○ Not built	<code>casestudy.manage</code>
9.8	Cross-tenant fleet analytics	○ Not built	Neither aggregates across organisations

10. Platform, deployment and integration

#	Feature	Status	Notes
10.1	Migration ledger and CLI	✔ Built	27 tables, head <code>0006_work_graph</code>
10.2	SQLite (dev) / MySQL (prod) from one DDL	✔ Built	
10.3	Multi-task safe with required-flag guards	✔ Built	<code>NXR_REQUIRE_DB/S3/REDIS</code>
10.4	ECS Fargate + RDS + ElastiCache + S3 + Neo4j on EC2	✔ Built	Documented in <code>AWS-ARCHITECTURE.md</code>
10.5	Health / readiness split for the load balancer	✔ Built	
10.6	Federated frontend (twin mounts inside a host shell)	✔ Built	Module federation — this is what makes WI-as-host feasible
10.7	Two codebases with two data layers	○ Not built	Twin uses raw SQL over <code>db/core.py</code> ; WI uses SQLAlchemy. One has to win, and it is the single biggest structural decision ahead
10.8	Org provisioning / connector management UI	○ Not built	Control app
10.9	CI, tests	✔ Built	528 tests passing
10.10	Notification delivery (email / push / SMS)	○ Not built	No transport in either codebase

11. Scorecard

Counted from the tables above, against the digital twin only:

Status	Count	Share
✔ Built in the digital twin	57	53%
○ Not built in the digital twin	51	47%
Total	108	

Just over half the platform is working here today. The twin is strongest on the plant itself, the dispatch loop, the operator and supervisor workspaces, the agents and the deployment story — sections 3, 7, 8 and 10 are nearly complete.

What the 51 are made of

Not all missing work costs the same, and the split matters for planning:

	Count	What it means
Already built in workforce-intelligence	32	Port or integrate — the design exists and runs
A narrower version exists here	6	Extend what is written: 1.6, 1.7, 3.9, 4.8, 5.1, 8.4
Genuinely new	13	Design and build from nothing

So of 51 gaps, **38 have something to start from** and 13 do not. Treating all 51 as new work would overstate the remaining effort by roughly a factor of three.

The thirteen with nothing to start from

#	Feature	Note
2.10	Manager multi-team view	The team view is scoped to one supervisor
3.11	Notifications	A model exists in WI; nothing delivers, in either
4.6	Clearance gates dispatch	The check that makes the competency model matter
4.7	Clearance expiry and recertification	
5.11	Curriculum / learning paths	
5.12	Skills matrix per person	
7.13	Twin access scoped by workforce role	
7.15	Graph population	Blocked on data, not code
8.8	Per-role agent access levels and budgets	
8.9	Reviewable agent decision log	
9.8	Cross-tenant fleet analytics	
10.7	Unified data layer	A decision, then a migration
10.10	Notification delivery transport	No email/push/SMS in either codebase

12. Suggested order

Sequenced by what unblocks the most, not by size.

First — decide the host and the data layer (10.7). WI as the application shell with the twin federated in is the stated direction and is already technically supported. The data layer is the real fork: the twin's raw-SQL store carries the migration ledger, MySQL/SQLite duality and 528 tests; WI's SQLAlchemy models

carry clearance, evidence, approvals and KPIs. Porting WI's 15 work models onto the twin's `db/` layer is the smaller and better-tested direction, but it is a decision to make deliberately rather than by drift.

Second — widen the role model (1.6, 1.7). Five roles and a full capability matrix. Everything in sections 4, 5 and 9 gates on this, and the twin's existing two-axis design (`role` for data, `persona` for job) extends to it cleanly.

Third — seed the graph (7.15). Nothing in section 7 demonstrates against 3 nodes, and this is a data task, not development.

Fourth — port the three management applications (2.3–2.5) onto the twin's dashboard standard, which is currently the better of the two.

Fifth — bring the scenario engine and builder across (section 6) with `twin_client.py` pointed at this repo's live twin API.

Then the genuinely new work, led by clearance gating (4.6), because it is small, it is the point of the competency model, and it belongs next to `assignable_user_ids`, which is already written and tested.